

Vasic-Digital-Reusable-Module-Suite

بناء مرة واحدة، وإعادة الاستخدام في كل مكان — أسطول من الوحدات الصغيرة المستقلة
Go و KMP والمنفصلة والمختبرة ذاتياً

الملاخص

عائلة كبيرة من الوحدات العامة القابلة لإعادة الاستخدام، منشورة تحت مساحات الأسماء
كل وحدة قائمة بذاتها، مختبرة digital.vasic.* (Go) و Kotlin Multiplatform. (Catalogizer)، ومستقلة الإصدار، وتستخدم كوحدة فرعية متساوية الكود في المنتجات الأكبر
والأسطول الأوسع). تجمع هذه الصفحة العديد من الأدوات الصغيرة التي قد تبدو غير HelixAgent، ذات أهمية لو عُرضت كل منها على حدة

وصف مختصر

المنفصلة — البنى التحتية الأساسية (المصادقة digital.vasic.* مجموعة منتقاة من وحدات
RAG، العملاء AI/التخزين المؤقت، قواعد البيانات، الإعدادات، قابلية الملاحظة)، لبنات بناء
فريق (LLM الوكالة، التخطيط)، وحاجز الحماية الدفاعي، MCP، التضمينات، VectorDB،
كل Kotlin Multiplatform. التوحيد القياسي) — بالإضافة إلى مجموعة مآة RedTeam،
منها عام، مختبر وقابل لإعادة الاستخدام

وصف مفصل

على رهان ميكلي واحد: فلسفة "دستور + العديد من الوحدات الفرعية vasic-digital تعتمد منظمة
، القابلة لإعادة الاستخدام والمنفصلة"، حيث لا يُكتب أي وظيفة عامة مرتين. فبدلاً من الأنظمة المتجانسة
تُستخرج كل وظيفة قابلة لإعادة الاستخدام إلى وحدة صغيرة مستقلة — مستودعها الخاص، اختباراتها
الخاصة، وثائقها الخاصة — وتُحافظ على انفصالها التام بحيث لا تتسرب أي تفاصيل خاصة بالمستهلك
إليها. تجمع هذه الصفحة هذه الوحدات لأنها، إذا عُرضت كل منها على حدة، ستكون بحجم مكتبة وقد
تبدو غير ذات أهمية كصفحة منتج مستقلة. لكن مجتمعةً، تشكل القوة الحقيقية المضاعفة للمنظمة
أصل هندسي خاص يحول مهمة "بناء منتج جديد" إلى "تجميع أجزاء مجربة"، كما تمثل الدعم
الملموس للدعاء بأن هذا الأسطول لا يعيد اختراع العجلة — بل يحافظ على عجلة جيدة واحدة
ويدحرجها في كل مكان

توفر الأساسيات التي (Go) البنى التحتية الأساسية. تتوزع المجموعة على ثلاث مجموعات رئيسية
(Redis/TTL)، التخزين المؤقت، (JWT/bcrypt) المصادقة، يحتاجها كل خدمة
الوسائط، الإعدادات، (المزوجة SQLite/PostgreSQL)، (الهجرات) قواعد البيانات

محدد المعدل، Prometheus/OpenTelemetry) قابلية الملاحظة، الوسيلة، ناقل الأحداث، Websocket مركز البث المباشر، S3/MinIO) التخزين، الأمان، التزامن، الاسترداد، http3، mdns، الاكتشاف، متعدد البروتوكولات) نظام الملفات، AI: rag، توفر الأساس لأنظمة (Go) العملاء/AI لبنات بناء وأكثر، التأخير، ضغط السياق اللانهائي) المحادثة، الذاكرة، embeddings، قاعدة بيانات المتجهات، سجل المهارات، مخطط الأدوات، MCP (Model Context Protocol)، (مصدر الأحداث، شجرة/HiPlan/MCTS التخطيط، (تنسيق سير العمل القائم على الرسوم البيانية) الوكالة عمليات، (SWE-bench/HumanEval/MMLU) المقارنة المرجعية، (الأفكار، نمذجة المكافأة/التعلم المعزز من التغذية) التحسين الذاتي، نماذج اللغة الكبيرة يوفر أدوات LLM حاجز الحماية الدفاعي. (تدوين الكائنات الموجه بالرموز) toon (الراجعة البشرية)، و التوحيد القياسي، (YAML مجموعات المواجهة المدفوعة بـ) RedTeam فريق: متانة المواجهة المتوازية نسخاً Kotlin Multiplatform كما تتضمن مجموعة. (توحيد المدخلات المواجهة) مكونات واجهة، KMP-الأمان، KMP-قواعد البيانات، KMP-مروءة للوحدات الأساسية (المصادقة، إلخ) للتطبيقات متعددة المنصات، KMP-المستخدم

المحتوى

لماذا أنشأناها

من الصفر (وغيرها Catalogizer، HelixAgent، Herald) إن إطلاق العديد من المنتجات في كل مرة يعد مضيعةً للموارد وغير متسق. فاستخلاص كل ما هو عام إلى وحدات مستقلة ومختبرة يعني أن الإصلاحات والتحسينات تنتشر عبر الأسطول بأكمله، وأن كل منتج جديد يُبنى من مكونات ماثبتة.

لماذا تُعد نقلة نوعية

وهي الطبقة التي لا تتاح — AI إنها في جوهرها “مكتبة معيارية خاصة” لبناء أنظمة خلفية مخصصة لـ لمعظم الفرق فرصة بنائها لأنها منشغلة بإعادة حل مسائل المصادقة والتخزين المؤقت والبنية التحتية لـ وحاجز الحماية الدفاعية لـ AI، للمرة الخامسة. هنا توجد البنى التحتية الأولية، ومكونات بناء RAG جاهزة للاستخدام ومستقلة الاختبار، وهذا ما يمكن فريقاً صغيراً من إطلاق أنظمة modules كـ LLM على مستوى المنتجات بوتيرة تتطلب عادةً فريقاً أكبر بكثير، دون تراكم الديون الناتجة عن التكرار التي عادة ما تتبع ذلك.

ما الجديد فيها

- تُعامل الوحدات الفرعية كمشاريع برمجية: (CONST-051) نظام الفصل الصارم عبر الأسطول متساوية، دون تحمل تفاصيل المستهلكين.
- MCP، التضمينات، (RAG، VectorDB، AI طبقة مخصصة للبنى الأولية لـ LLMops)، الوكالة، التخطيط، ToolSchema،
- لتعزيز المتانة في (LLM (RedTeam، Normalize مجموعة حواجز حماية دفاعية لـ مواجهة الهجمات.
- تشارك في نفس الأعراف Kotlin Multiplatform و Go مجموعات وحدات متوازية لـ

التحديات والحلول

- تم حلها من خلال عقد الفصل في الدستور وحقن: تجنب تدهور الترابط بين عشرات الوحدات. تفاصيل المستهلكين في وقت التشغيل.
- تم حلها من خلال اعتماد أعراف مشتركة: الحفاظ على اتساق واختبار العديد من الوحدات HelixConstitution. وبنية حوكمة (اختبارات/وثائق/تحديات لكل وحدة).
- Kotlin Multiplatform تم حلها من خلال مرآة: الوصول عبر المنصات المختلفة للوحدات الأساسية.

مجموعة التقنيات (السبب وكيفية الاستخدام)

- **Go** — (digital.vasic.*). معظم الوحدات
- **Kotlin Multiplatform** — (المصادقة/قواعد البيانات KMP). الأمان/واجهة المستخدم/الترامن/محدد معدل
- **Redis / PostgreSQL / SQLite** — وحدات التخزين المؤقت وقواعد البيانات والتخزين الأولية
- **Prometheus / OpenTelemetry** — وحدة المراقبة
- **WebSocket / HTTP/3 (quic-go) / mDNS** — وحدات الشبكات
- **Vector DB / embeddings / RAG / MCP** — AI وحدات البنى الأولية لـ
- **YAML** — الهجومية RedTeam نماذج وتكوينات

“SCAFFOLD / WIP” غير مُتحقق / قيد التطوير: عدة مستودعات تنظيمية مُعلّمة كـ (مثل PliniusCommon، I-LLM، HyperTune، AutoTemp، Veritas، Ouroborous، Claritas، LeakHub، GandalfSolutions). عرضها كمراحل أولية أو. هياكل عمل، وليست جاهزة للإطلاق